# Sistema de Punto de Venta para Comercios Locales

Entrega integrada — Fases 1 y 2

Entrega académica

Octubre de 2026

## Índice


**1 Fase 1 — Análisis, modelado y requisitos** **3**

**2 Fase 1 — Sistema de Punto de Venta para Comercios Locales** **3**
2.1 Problema y objetivo . . . 3
2.2 Alcance acordado . . . 3
2.3 Requerimientos funcionales y reglas verificables . . . 3
2.4 Requerimientos técnicos . . . 4
2.5 Decisiones y límites . . . 5
2.6 Para la defensa . . . 5

**Diagrama de clases UML** **6**

**3 Diccionario de clases del dominio** **7**
3.1 Relaciones del UML . . . 8
3.2 Para la defensa . . . 8

**4 Pantallas previstas para el sistema** **8**

**5 Fase 2 — Diseño e implementación** **10**
5.1 Requisitos funcionales implementados . . . 10
5.2 Requisitos técnicos . . . 10

**6 Fase 2 — Lógica, JDBC y MVC** **10**
6.1 Recorrido para estudiar . . . 10
6.2 Para la defensa: modelo y colecciones . . . 11
6.3 Para la defensa: JDBC y transacciones . . . 11
6.4 Para la defensa: MVC, permisos y excepciones . . . 11
6.5 Alcances de seguridad académica . . . 12

**7 Modelo entidad-relación** **12**
7.1 Integridad y acciones referenciales . . . 12
7.2 Para la defensa . . . 13

**8 Ejecución — guía breve (una página)** **13**

**9 Evidencia de validación de las fases 1 y 2** **14**
9.1 Entorno ejecutado . . . 14
9.2 Comprobaciones realizadas . . . 14
9.3 Cómo reproducir . . . 15
9.4 Límites . . . 15

**10 Evidencia de CRUD — video de 1-2 minutos** **15**
10.1 Demostración adicional de ventas (fuera del video de CRUD) . . . . . 16
10.2 Para la defensa . . . . . 16

**11 Anexo A — Script de creación de tablas** **16**

# 1 Fase 1 — Análisis, modelado y requisitos

# 2 Fase 1 — Sistema de Punto de Venta para Comercios Locales

## 2.1 Problema y objetivo

Un comercio que registra ventas e inventario manualmente puede vender existencias agotadas, perder el historial de precios y carecer de responsables identificables. El sistema centraliza productos, categorías, clientes, usuarios y ventas; descuenta inventario de forma atómica y permite consultar transacciones por fecha y responsable.

## 2.2 Alcance acordado

Aplicación de escritorio Java/Swing, MySQL y JDBC, organizada en MVC. Fase 2: lógica y vista de consola verificable; Fase 3: Swing. Sin proveedores, compras, devoluciones, facturación fiscal ni reportes avanzados. El precio incluye IVA: no se calcula ni desglosa el impuesto. Se registra el método de pago; no se procesa una tarjeta ni se almacenan sus datos.

Actores: administrador (todos los módulos) y cajero (consulta de productos, Caja, clientes desde Caja e historial propio). El administrador ve todas las ventas. La aplicación no usa una conexión MySQL distinta por persona; autentica usuarios de negocio con BCrypt.

## 2.3 Requerimientos funcionales y reglas verificables

| ID | Requerimiento / regla | Criterio de aceptación |
|---|---|---|
| RF01 | Autenticar usuarios activos | Clave incorrecta, usuario inexistente o inactivo rechaza el acceso con mensaje genérico. |
| RF02 | Aplicar permisos en controladores | Un cajero no puede gestionar usuarios, categorías ni productos, aunque intente invocar la operación directamente. |
| RF03 | CRUD de categorías | Nombre obligatorio y único; borrar una categoría con productos falla, incluso con productos inactivos. |
| RF04 | CRUD y búsqueda de productos | Código único, nombre obligatorio, precio positivo con máximo 2 decimales, stock y mínimo enteros no negativos, categoría existente. |
| RF05 | Desactivar productos vendidos | Sin ventas se permite DELETE; con ventas se conserva la fila y se marca activo=false. |
| RF06 | Formar carrito con ArrayList | Agregar, agrupar el mismo producto, consultar y quitar; cantidades enteras positivas y sin exceder existencias. |

| ID | Requerimiento / regla | Criterio de aceptación |
|---|---|---|
| RF07 | Confirmar venta atómicamente | Usuario activo, cliente existente y al menos un detalle; encabezado, detalles y stock se confirman juntos o se revierten juntos. |
| RF08 | Evitar sobreventa concurrente | Dos cajas que compiten por la última unidad producen una sola venta exitosa. |
| RF09 | Perecederos | Una fecha de vencimiento por producto. No vender después de esa fecha; de 0 a 3 días inclusive se descuenta 10 %, con HALF_UP a 2 decimales. |
| RF10 | Conservar precio histórico | Cada detalle guarda el precio aplicado y subtotal; cambiar el catálogo no modifica importes históricos. |
| RF11 | Consultar historial inclusivo por fechas | Administrador ve todas las ventas; cajero solo las propias; se muestran sus detalles. |
| RF12 | Alertar stock bajo | Mostrar alerta cuando stock <= stockMinimo. |
| RF13 | Gestionar clientes | Ambos roles pueden registrar y editar clientes desde Caja; solo administrador elimina clientes sin ventas. Consumidor Final (NIT CF) no se modifica ni elimina. |
| RF14 | Gestionar usuarios | Administrador crea, consulta, edita y elimina; cuentas con ventas se desactivan. No puede borrar su cuenta ni quitarse su propio acceso administrativo. |

## 2.4 Requerimientos técnicos

- Java camelCase y SQL snake_case; paquetes `model`, `view`, `controller`, `dao`, `util` bajo `com.progra.pos`.
- Modelos encapsulados con atributos privados; constructores validan invariantes; modificación del catálogo mediante nuevos objetos y UPDATE explícito.
- Herencia y polimorfismo en usuarios y productos. ArrayList en carrito, detalles y resultados DAO.
- Todo SQL de la aplicación usa PreparedStatement; entradas del usuario mediante parámetros. Fragmentos SQL condicionales solo provienen de constantes internas.
- JDBC con try-with-resources. SQLException se traduce a AccesoDatosException; la vista muestra un mensaje sin exponer consultas ni credenciales. Rollback conserva la excepción original.
- Contraseñas BCrypt, factor 12 y salt por hash. Mínimo 8 caracteres y máximo 72 bytes UTF-8; no se truncan silenciosamente.
- BigDecimal ↔ DECIMAL(12,2); LocalDate ↔ DATE; LocalDateTime ↔ DATETIME. Reloj local del comercio: no mezclar zonas horarias entre cajas.
- No se guardan contraseñas de MySQL en Git. Variables POS_DB_URL, POS_DB_USER, POS_DB_PASSWORD.

## 2.5 Decisiones y límites

`Venta.usuario` referencia Usuario, no exclusivamente Cajero: el administrador también vende. `Venta` en construcción puede estar vacía; el DAO no guarda ventas vacías. No hay edición ni eliminación de ventas confirmadas. La composición es conceptual y no exige borrado SQL en cascada: aquí se protege la auditoría con RESTRICT.

El carrito muestra un total estimado; al confirmar acepta precios vigentes, calculados nuevamente dentro de la transacción. Si cambia el precio mientras se prepara el carrito, el importe definitivo puede cambiar. El historial conserva precio y cantidad, pero muestra los nombres actuales de producto y cliente: no es un comprobante fiscal inmutable.

Se admite una sola fecha de vencimiento por código. Lotes con fechas diferentes requieren una ampliación fuera del alcance. El NIT se valida como texto obligatorio y único, sin validación tributaria oficial. No hay recuperación de contraseña, MFA ni límite de intentos: alcance académico local.

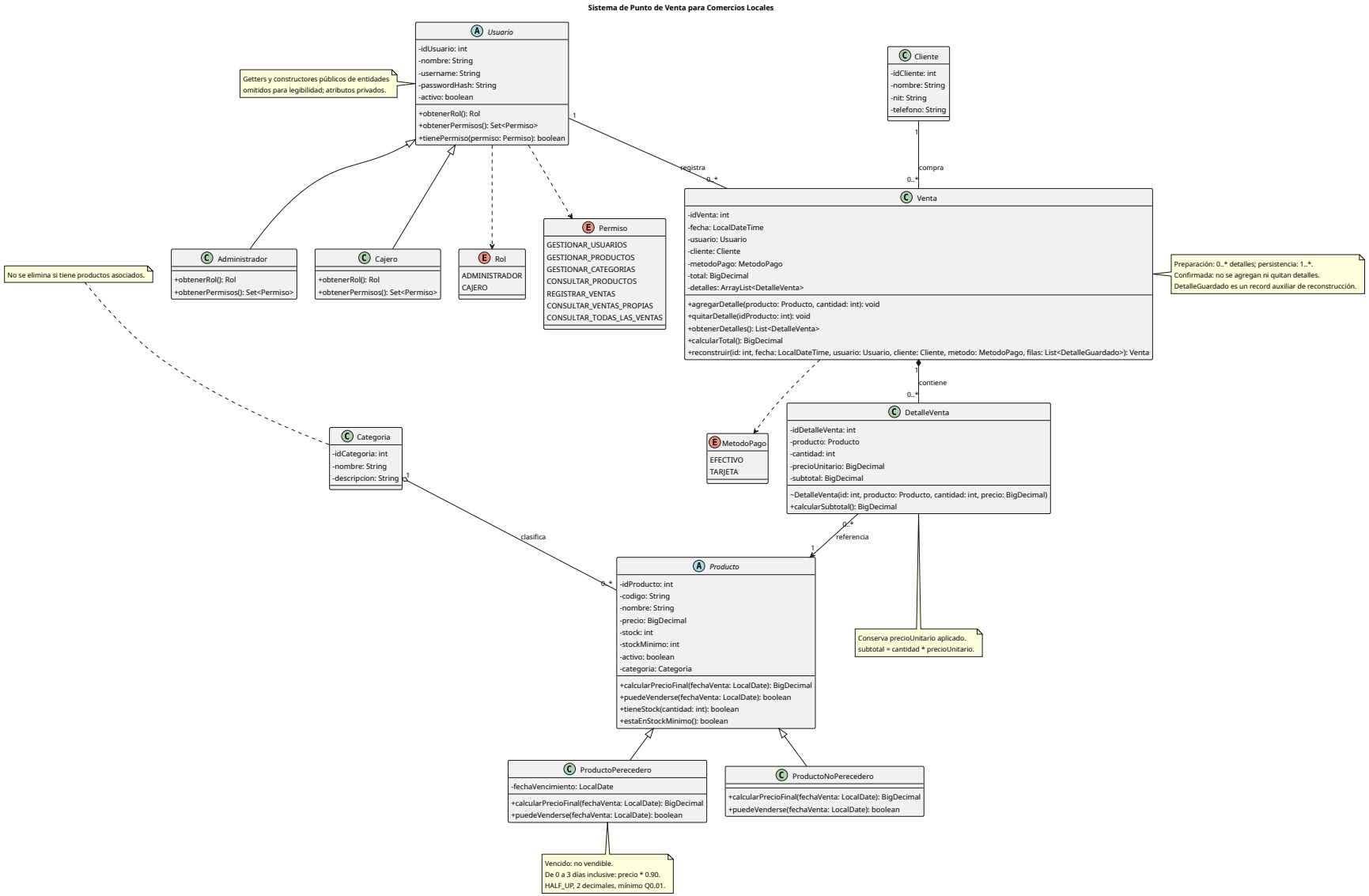
## 2.6 Para la defensa

**Explicación:** una regla debe indicar qué puede ocurrir y cómo demostrar que se cumple. Las validaciones del modelo previenen entradas inválidas; las restricciones SQL protegen los datos; la transacción protege el conjunto de cambios.

1. **¿Por qué validar en Java y SQL?** Java ofrece mensajes tempranos; SQL impide guardar datos inválidos incluso si otro cliente accede a la BD.
2. **¿Por qué Usuario en Venta?** Ambos roles pueden registrar ventas, y el historial necesita identificar al responsable real.
3. **¿El stock del carrito garantiza la venta?** No. Otra caja puede comprar antes; se verifica nuevamente con bloqueo dentro de la transacción.

**Cambio en vivo: mostrar alerta tres unidades antes del mínimo.** 1) Abrir Producto.java. 2) Cambiar la comparación en estaEnStockMinimo a `(long) stock <= (long) stockMinimo + 3`. 3) Actualizar RF12. 4) Probar mínimo=5 con stock 8 y 9; solo 8 debe alertar. Convertir a long evita desbordamiento al sumar.

# Diagrama de clases UML



# 3 Diccionario de clases del dominio

Ruta de las clases: `src/main/java/com/progra/pos/model/`.

| Clase | Responsabilidad y atributos principales | Métodos clave |
|---|---|---|
| Usuario (abstracta) | Identidad y autenticación: idUsuario:int, nombre/username/passwordHash:String, activo:boolean. | obtenerRol, obtenerPermisos, tienePermiso. |
| Administrador | Especialización de Usuario con todos los permisos. | Sobrescribe obtenerRol y obtenerPermisos. |
| Cajero | Especialización de Usuario limitada a consulta, Caja e historial propio. | Sobrescribe obtenerRol y obtenerPermisos. |
| Categoria | Agrupación: idCategoria:int, nombre/descripcion:String. | Constructor validado, getters. |
| Producto (abstracta) | Catálogo: idProducto:int; codigo/nombre:String; precio:BigDecimal; stock/stockMinimo:int; activo:boolean; categoria:Categoria. | calcularPrecioFinal, puedeVenderse, tieneStock, estaEnStockMinimo. |
| ProductoPerecedero | Agrega fechaVencimiento:LocalDate; valida vigencia y descuento. | Sobrescribe calcularPrecioFinal y puedeVenderse. |
| ProductoNoPerecedero | Mantiene precio base y exige estar activo. | Sobrescribe calcularPrecioFinal y puedeVenderse. |
| Cliente | Comprador: idCliente:int, nombre/nit/telefono:String. | Constructor validado, getters. |
| Venta | Encabezado: idVenta:int, fecha:LocalDateTime, usuario:Usuario, cliente:Cliente, metodoPago:MetodoPago, total:BigDecimal, detalles:ArrayList. | agregarDetalle, quitarDetalle, calcularTotal, obtenerDetalles, reconstruir. |
| DetalleVenta | Línea: idDetalleVenta:int, producto:Producto, cantidad:int, precioUnitario/subtotal:BigDecimal. | calcularSubtotal; constructor de paquete coordinado por Venta. |
| Rol (enum) | ADMINISTRADOR, CAJERO. | valueOf, name heredados de Enum. |
| Permiso (enum) | Capacidades explícitas de cada rol. | Valores usados por Sesion.exigir. |
| MetodoPago (enum) | EFECTIVO, TARJETA. | valueOf, name. |

`Venta.DetalleGuardado` es un record auxiliar de reconstrucción JDBC; no es otra tabla. Transporta id:int, producto:Producto, cantidad:int y precio:BigDecimal para restaurar el precio histórico. Venta confirmada no permite agregar ni quitar detalles. Las listas devueltas son copias no modificables.

### 3.1 Relaciones del UML

- Herencia: Administrador/Cajero → Usuario y Perecedero/NoPerecedero → Producto (`extends`).
- Agregación: Categoria 1 ◇— 0..* Producto; cada producto tiene exactamente una categoría y puede reasignarse.
- Composición: Venta 1 ◆— 0..* DetalleVenta durante preparación; al persistir exige 1..*. Cada detalle tiene una única venta propietaria.
- Asociación: Usuario 1 — 0..* Venta y Cliente 1 — 0..* Venta.
- Asociación navegable: DetalleVenta 0..* → 1 Producto.

### 3.2 Para la defensa

**Explicación:** encapsular no exige crear setters para todo. Los objetos del catálogo son inmutables; editar significa construir una versión validada y enviarla al DAO. Venta sí cambia durante la preparación y controla sus detalles.

1. **¿Por qué Producto es abstracta?** No hay un producto genérico vendible sin decidir su comportamiento respecto del vencimiento.
2. **¿Dónde se usa ArrayList?** En detalles de Venta, líneas de VentaController y listas devueltas por los DAO.
3. **¿Por qué obtenerDetalles no devuelve la lista interna?** Para impedir que un llamador agregue líneas sin validar cantidad, precio y stock.

**Cambio en vivo: descuento de 15 % hasta cinco días antes.** 1) Abrir ProductoPerecedero.java. 2) Cambiar `dias <= 3` por `dias <= 5`. 3) Cambiar `0.90` por `0.85`. 4) Actualizar ModeloTest con días 0, 5, 6 y -1; Q100 debe resultar Q85, Q85, Q100 y rechazo. 5) Actualizar RF09 y la nota UML.

## 4 Pantallas previstas para el sistema

La Fase 1 deja definido el alcance visual esperado. La implementación de escritorio se incorporó en la Fase 3 con Java Swing; esta lista documenta qué verá cada rol y cómo se relacionan las pantallas con los requisitos.

| Pantalla | Usuario | Funciones previstas | Requisitos relacionados |
|---|---|---|---|
| Inicio de sesión | Administrador y cajero | Identificar usuario activo y abrir el menú según permisos. | RF01–RF02 |
| Menú principal | Administrador y cajero | Navegar únicamente a módulos autorizados. | RF02 |
| Productos | Administrador; consulta para cajero | Buscar y mantener productos, categoría, precio, existencia, mínimo y vencimiento; alertar existencia baja. | RF04–RF05, RF09, RF12 |
| Categorías | Administrador | Crear, consultar, editar y borrar categorías sin productos asociados. | RF03 |

| Pantalla | Usuario | Funciones previstas | Requisitos relacionados |
|---|---|---|---|
| Caja / nueva venta | Administrador y cajero | Elegir cliente y método de pago; agregar, agrupar y quitar productos; revisar total y confirmar venta. | RF06–RF10, RF13 |
| Clientes desde Caja | Administrador y cajero | Registrar y editar clientes durante el cobro; aplicar restricciones para Consumidor Final y borrado. | RF13 |
| Historial de ventas | Administrador y cajero | Filtrar por fechas; administrador ve todas, cajero solo propias; consultar detalles. | RF11 |
| Usuarios | Administrador | Crear, consultar, editar y eliminar o desactivar cuentas según historial. | RF14 |

El producto acordado es una aplicación de escritorio Swing, no una página web. La consola de Fase 2 permite probar la lógica; la interfaz gráfica es la fase posterior.

# 5 Fase 2 — Diseño e implementación

La Fase 2 implementa las reglas funcionales definidas en Fase 1 mediante Java, JDBC, MySQL y una arquitectura MVC. Incluye autenticación y permisos, CRUD de categorías, productos, clientes y usuarios, carrito de venta, control de inventario, registro transaccional e historial. La interfaz de esta fase es de consola; la interfaz gráfica Swing se añadió en Fase 3.

## 5.1 Requisitos funcionales implementados

- Autenticar usuarios activos y aplicar permisos desde los controladores.
- Registrar y mantener categorías, productos, clientes y usuarios con las validaciones del dominio.
- Mantener un carrito con productos agrupados, precios calculados y existencias verificadas.
- Confirmar venta con transacción atómica: encabezado, detalles y descuento de inventario se confirman juntos o se revierten juntos.
- Evitar sobreventa concurrente mediante bloqueos de fila y nueva validación de existencias al confirmar.
- Conservar el precio aplicado en cada detalle y consultar historial por fechas, con visibilidad según rol.
- Aplicar vencimiento y descuento a productos perecederos, y alertar cuando el stock llega al mínimo.

## 5.2 Requisitos técnicos

- JDK 21 o superior y Maven; la validación documentada utilizó JDK 25.0.4.1 con compilación `release 21`.
- MySQL 8.0.16 o superior (se probó MySQL Community Server 8.4.11) y Connector/J gestionado por Maven.
- JDBC con `PreparedStatement`, `try-with-resources`, transacciones y excepciones de acceso traducidas para la aplicación.
- Contraseñas del POS almacenadas con BCrypt; credenciales de MySQL configuradas con `POS_DB_URL`, `POS_DB_USER` y `POS_DB_PASSWORD`, sin incluirlas en Git.
- Valores monetarios con `BigDecimal` y `DECIMAL(12,2)`; fechas con tipos Java y SQL correspondientes.

# 6 Fase 2 — Lógica, JDBC y MVC

## 6.1 Recorrido para estudiar

1. `model/Producto.java` y sus hijas: invariantes y polimorfismo. No conocen JDBC ni componentes gráficos.
2. `model/Venta.java` y `DetalleVenta.java`: ArrayList, suma con BigDecimal y conservación de importes históricos.
3. `util/Conexion.java`, `Validacion.java`, `PasswordUtil.java`, `AccesoDatosException.java`: conexión por variables, reglas compartidas, BCrypt y traducción de errores.
4. `dao/CategoriaDao.java`: ejemplo pequeño de INSERT, SELECT, UPDATE y DELETE con PreparedStatement y try-with-resources.
5. `dao/ProductoDao.java`, `ClienteDao.java`, `UsuarioDao.java`: CRUD, discriminadores y borrados condicionados por historial.
6. `controller/AutenticacionController.java`, `Sesion.java`, `CatalogoController.java`: autenticación y permisos comprobados en cada operación. Sesion conserva solo el id y consulta estado/rol actual.
7. `controller/VentaController.java`, `dao/VentaDao.java`: carrito, confirmación atómica e historial con filtro por usuario.
8. `view/ConsolaApp.java`: entrada/salida y captura de errores; no contiene SQL ni llama DAO directamente. Fase 3 añadirá vistas Swing que reutilicen controladores.

Todas las rutas son relativas a `src/main/java/com/progra/pos/`. Las pruebas están en `src/test/java/com/progra/pos/`.

## 6.2 Para la defensa: modelo y colecciones

**Explicación:** ArrayList mantiene el carrito y sus líneas; se busca por id y se reemplaza la línea al aumentar cantidad. Las copias no modificables impiden saltarse las reglas.

1. **¿Alta, baja y búsqueda en memoria?** VentaController.agregar añade o reemplaza por id; quitar usa removeIf; el bucle de agregar busca la línea existente.
2. **¿Por qué BigDecimal y no double?** BigDecimal representa decimales monetarios sin errores binarios; se construye desde String y se redondea explícitamente.
3. **¿El polimorfismo necesita instanceof?** El cálculo de precio no: se invoca sobre Producto. El DAO sí distingue la subclase al guardar sus datos específicos.

**Cambio en vivo: prohibir cantidades mayores a 100 por línea.** 1) Validar cantidad acumulada en Venta.agregarDetalle. 2) Repetir límite temprano en VentaController.agregar. 3) Mantener la verificación final del DAO por medio de Venta. 4) Probar 100, 101 y dos adiciones 60+50. 5) Documentar la regla.

## 6.3 Para la defensa: JDBC y transacciones

**Explicación:** confirmar no es un conjunto de consultas independientes. Se abre una conexión, se desactiva autocommit, se bloquea el usuario y después los productos en orden de id, se valida y actualiza, se insertan venta y detalles y se hace commit. Si falla cualquier paso se ejecuta rollback.

1. **¿Por qué SELECT FOR UPDATE?** Retiene bloqueos hasta commit/rollback e impide que dos transacciones descuenten simultáneamente la misma existencia como si fuera la última disponible.
2. **¿Por qué ordenar productos?** Mantener un orden común reduce interbloqueos. No los elimina completamente; si ocurren se informa el fallo y el usuario puede volver a intentar.
3. **¿Cómo evita inyección PreparedStatement?** Separa la estructura SQL de los valores; el texto introducido se trata como dato, no como instrucciones.

**Cambio en vivo: venta solo con stock final >= 2.** 1) En VentaDao confirmar, antes del UPDATE, comprobar `(long) producto.getStock() - cantidad >= 2`. 2) Mantenerlo dentro del bloqueo. 3) Añadir prueba con stock 3/cantidad 1 (acepta) y stock 2/cantidad 1 (rechaza). 4) Comprobar que rollback no descuenta otros productos. Esta política es distinta de la alerta de stock mínimo.

## 6.4 Para la defensa: MVC, permisos y excepciones

**Explicación:** la vista transforma entrada textual en datos y muestra mensajes. El controlador autoriza y coordina. El modelo valida su estado. El DAO trabaja con MySQL. Ocultar un botón será una ayuda visual, no la autorización real.

1. **¿Dónde se atrapa SQLException?** En DAO; se envuelve conservando causa. La vista captura AccesoDatosException y muestra un mensaje sin imprimir credenciales ni SQL.
2. **¿Qué ocurre si desactivan a un usuario conectado?** Sesion.exigir consulta otra vez su estado; además la confirmación de venta comprueba al usuario bloqueado dentro de su transacción.
3. **¿Swing debe ejecutar SQL desde un botón?** No. El evento llamará al controlador y la operación de BD deberá ejecutarse fuera del hilo de eventos con SwingWorker.

**Cambio en vivo: cajero sin acceso a historial.** 1) Quitar CONSULTAR_VENTAS_PROPIAS de Cajero.obtenerPermisos. 2) Mantener las comprobaciones existentes. 3) Ocultar la opción de consola/Swing para ese rol. 4) Probar llamada al controlador, que debe rechazar incluso sin botón. 5) Actualizar RF02/RF11 y pruebas.

### 6.5 Alcances de seguridad académica

El control de permisos protege las rutas de la aplicación. Quien conoce la contraseña MySQL o ejecuta código arbitrario con acceso a DAO puede saltarse la capa de control: las credenciales de BD no deben compartirse con usuarios finales. La creación inicial de administrador es una operación local de instalación, solo con la tabla usuarios vacía; ejecutarla una sola vez, sin instalaciones concurrentes. El bloqueo evita autoeliminación del administrador, pero no implementa coordinación global entre varios administradores que se desactiven mutuamente. No es un producto listo para Internet.

## 7 Modelo entidad-relación

Las propiedades Java camelCase se traducen a columnas snake_case. Usuario.usuarioId no existe: la PK Java es `idUsuario`; `Venta.usuario` se representa con `ventas.usuario_id`. Las dos jerarquías usan una tabla por jerarquía: `usuarios.rol` y `productos.tipo` discriminan la subclase al leer JDBC.

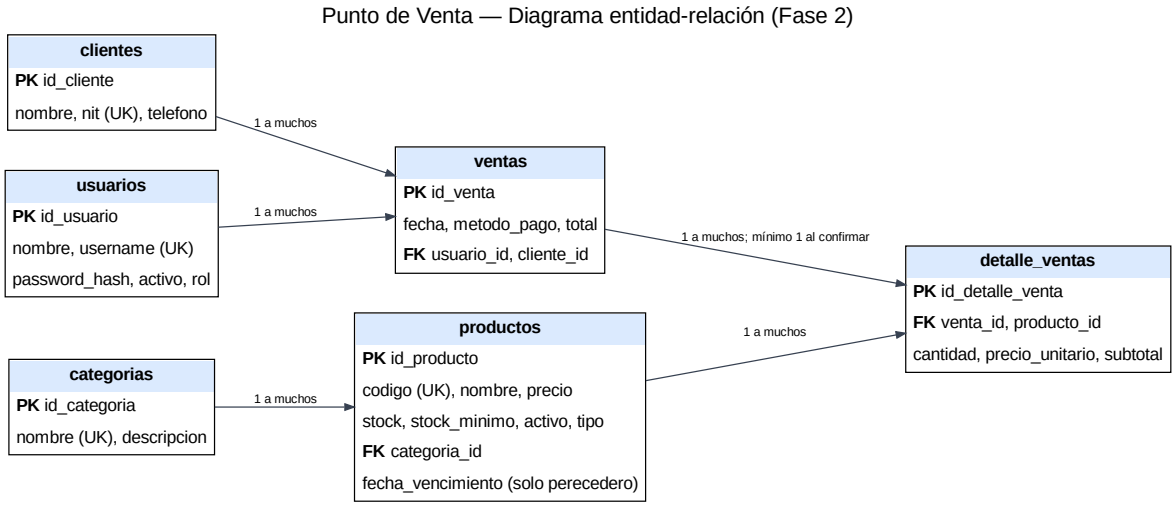



Figura 1: Diagrama entidad-relación del sistema

En el diagrama, `DECIMAL(12_2)` representa `DECIMAL(12,2)` del SQL: doce dígitos, dos de ellos decimales. Todas las columnas son NOT NULL excepto fecha_vencimiento. Las descripciones y teléfonos admiten cadena vacía.

### 7.1 Integridad y acciones referenciales

- Todas las tablas usan InnoDB para soportar transacciones y bloqueos por fila.
- Todas las FK tienen ON DELETE RESTRICT y ON UPDATE RESTRICT. Las PK son identidades internas que no se editan; no se propagan cambios ni borrados del historial.
- Categoria con productos: no se elimina. Producto con detalles: el DAO desactiva. Usuario con ventas: el DAO desactiva. Cliente con ventas: la FK impide borrar.
- UNIQUE: username, nombre de categoría, código, NIT y pareja (venta_id, producto_id). El carrito agrupa líneas del mismo producto.
- CHECK: precios positivos; stock y mínimo no negativos; cantidad positiva; subtotal correcto; rol, método y tipo válidos; vencimiento obligatorio exclusivamente para perecederos.

- El SQL no garantiza por sí solo que una venta tenga detalles ni que total sea su suma. VentaDao los guarda juntos, calcula total y rechaza carritos vacíos. El usuario de BD de la aplicación debe ser la vía controlada de escritura.
- No hay DELETE de ventas en la aplicación. Composición no equivale automáticamente a CASCADE.

### 7.2 Para la defensa

**Explicación:** las clases modelan comportamiento y la BD conserva estado. Una subclase no necesita obligatoriamente una tabla propia; un discriminador permite reconstruirla sin duplicar atributos comunes.

1. **¿Cómo sabes qué subclase crear al leer productos?** ProductoDao revisa tipo; para PERECEDERO lee también fecha_vencimiento.
2. **¿Por qué RESTRICT en detalles si hay composición?** El proyecto conserva auditoría y no permite borrar ventas confirmadas; la propiedad del detalle no obliga a habilitar borrado físico.
3. **¿Un CHECK puede comprobar la suma de otra tabla?** No en este esquema MySQL. Lo garantiza la operación transaccional de VentaDao y la lectura verifica la consistencia del total.

**Cambio en vivo: agregar pago por transferencia.** 1) Agregar TRANSFERENCIA a MetodoPago. 2) Ampliar ck_venta_pago mediante una migración ALTER TABLE, no reimportar el esquema sobre datos existentes. 3) Actualizar el prompt de ConsolaApp y el E-R. 4) Guardar una venta con ese método y comprobar el historial. La cadena cabe en VARCHAR(10)? No: TRANSFERENCIA tiene 13 caracteres; ampliar a VARCHAR(20) también.

## 8 Ejecución — guía breve (una página)

**Requisitos:** NetBeans 31, JDK completo 21 o superior (tu JDK 25 es compatible con release 21), Maven 3.9+, MySQL 8.4. Connector/J y BCrypt se descargan con Maven. Confirmar servidor con `SELECT VERSION();`; “26.7” queda pendiente de identificar.

**NetBeans:** Archivo → Abrir proyecto → seleccionar la carpeta que contiene pom.xml. Elegir un JDK completo en Herramientas → Plataformas Java y en las propiedades del proyecto. Ejecutar Clean and Build. Abrir `src/main/java/com/progra/pos/view/ConsolaApp.java` y ejecutar el proyecto usando su main. Si la acción Maven no configura clase principal, usar el objetivo `compile exec:java -Dpos.mainClass =com.progra.pos.view.ConsolaApp`.

**Base de datos local propia:** crear una BD vacía `pos_dev` con utf8mb4; ejecutar `sql/01_schema.sql` una sola vez desde MySQL Workbench o el cliente mysql. Crear una cuenta de aplicación dedicada. Para uso normal basta SELECT, INSERT, UPDATE y DELETE sobre esa BD. El esquema crea Consumidor Final y no crea contraseñas compartidas.

**Conexión:** configurar en el proceso que inicia NetBeans `POS_DB_URL`, `POS_DB_USER`, `POS_DB_PASSWORD` y reiniciar el IDE. Ejemplo URL: `jdbc:mysql://localhost:3306/pos_dev?sslMode=VERIFY_IDENTITY`. Esa opción requiere certificado confiable válido para el hostname; configurar truststore Connector/J si la CA es privada. No guardar valores secretos en Git ni en capturas. Usuario MySQL y usuario del POS son cuentas distintas.

**Administrador inicial:** en una terminal con las variables configuradas ejecutar `mvn compile exec :java -Dpos.mainClass=com.progra.pos.view.ConsolaApp -Dexec.args=--crear-admin`; introducir nombre, username y una contraseña propia de al menos 8 caracteres. Solo funciona con usuarios vacío. Después ejecutar `mvn compile exec:java -Dpos.mainClass=com.progra.pos.view.ConsolaApp`, iniciar sesión y crear un cajero con `usuario-crear`. Credenciales de prueba: las que acabas de crear; no hay claves fijas en el repositorio.

**Alternativa Linux/Docker incluida:** desde la raíz ejecutar `scripts/mysql-local.sh`, luego `scripts/run.sh -Dexec.args=--crear-admin` y después `scripts/run.sh`. El script usa MySQL local en puerto 3307, genera claves en `.local/mysql.env` ignorado por Git y confía en la CA local mediante un truststore. JDK completo, Docker, Python >= 3.12 y curl deben estar disponibles; `scripts/mvn.sh` descarga Maven si falta. No eliminar el contenedor si necesitas sus datos; detener con `docker stop progra-mysql-fase2` y reiniciar con mysql-local.sh. El script preserva el esquema existente y no aplica migraciones.

**Pruebas:** `mvn test` ejecuta modelo; las pruebas MySQL se omiten si no habilitas la BD. En Linux/Docker ejecutar `scripts/test-mysql.sh` para ambas suites contra `pos_test`, exclusivamente de pruebas: borra sus filas antes de cada caso. No apuntar a datos que deban conservarse. Compilar antes de demo y revisar el resumen de Surefire en target/surefire-reports. El detalle de resultados reales está en validacion.md.

**Errores habituales:** versión release no soportada → elegir JDK completo, no JRE; conexión rechazada → comprobar servicio/puerto; TLS → configurar certificado/CA; duplicado o FK → revisar código, nombre o referencias; Maven en nube con proxy → scripts/mvn.sh genera configuración local sin publicar credenciales. La consola muestra errores de entradas sin cerrar el menú; el login fallido termina y requiere ejecutar nuevamente.

# 9 Evidencia de validación de las fases 1 y 2

Documento de la entrega anterior. La interfaz Swing y las pruebas adicionales están en validación de Fase 3.

## 9.1 Entorno ejecutado

- Linux; OpenJDK Temurin 25.0.4.1, compilación con release 21.
- Maven 3.9.11, Connector/J 9.3.0, jbcrypt 0.4.
- MySQL Community Server 8.4.11 en contenedor mysql:8.4, puerto local 3307.
- TLS con verificación de CA local mediante truststore PKCS12.
- pos_dev para la aplicación; pos_test desechable para pruebas. Las pruebas rechazan URLs que no nombren exclusivamente pos_test.

## 9.2 Comprobaciones realizadas

| Comprobación | Resultado |
|---|---|
| ModeloTest | 8 pruebas, 0 fallos, 0 errores, 0 omitidas. |
| MySqlTest | 12 pruebas, 0 fallos, 0 errores, 0 omitidas. |
| Compilación y empaquetado | Correctos; target/punto-venta-1.0.0.jar. Ejecutar con Maven para incluir dependencias; no es un JAR autónomo. |
| Consola | Login, alta de categoría, consultas, historial, resumen vacío, rechazo de entrada y salida verificados con BD de pruebas. |
| Scripts MySQL | Arranque y repetición correctos; no se reimporta un esquema existente. |
| UML | PlantUML validó sintaxis y generó diagrama-clases.svg. |
| Guía | PDF con fuentes incrustadas, una página A4, inspeccionado visualmente. |

Las pruebas JDBC ejercitan CRUD de cuatro entidades, unicidad, búsqueda literal, login inválido, inyección como texto, permisos, desactivación de sesión, protección de CF y de la propia

cuenta administradora, precio histórico, descuento, vencimiento, rollback, carrito agrupado, confirmación fallida sin vaciar carrito y CHECK de MySQL.

La prueba concurrente usa dos conexiones y dos cajeros que compiten por una unidad: exige exactamente un éxito, stock cero y una venta. La prueba de rollback exige que, al fallar el segundo producto, las existencias del primero permanezcan intactas y no exista venta parcial.

### 9.3 Cómo reproducir

Desde la raíz con JDK completo y Docker:

```
scripts/mysql-local.sh
scripts/test-mysql.sh
scripts/mvn.sh -DskipTests package
```

El primer script crea o reinicia el contenedor y prepara CA/BD. El segundo configura la BD desechable y ejecuta ambas suites. Reportes: target/surefire-reports/. El último empaqueta sin repetir pruebas; no es otra ejecución de tests.

Sin POS_RUN_DB_TESTS=true, mvn test solo demuestra el modelo: MySQL se omite. No presentar ese resultado como validación de JDBC.

### 9.4 Límites

No se ejecutó NetBeans ni una interfaz gráfica: no están instalados en esta nube. Verificar NetBeans 31 en el equipo del estudiante siguiendo ejecucion.pdf. No se probó exactamente JDK 25.0.3: se usó 25.0.4.1 con bytecode Java 21. “MySQL 26.7” sigue pendiente de identificar; el esquema requiere MySQL >=8.0.16 y está probado en 8.4.11. No se desarrolló Swing ni se grabó video: la interfaz corresponde a Fase 3 y se entrega el guion de evidencia.

## 10 Evidencia de CRUD — video de 1–2 minutos

Preparación fuera de grabación: MySQL iniciado, proyecto compilado, administrador y cajero creados, consola autenticada. No mostrar contraseñas ni archivos `.local`. Para que quepa en dos minutos ensaya los comandos. Usa códigos únicos para no chocar con datos anteriores; anota los ids devueltos.

| Tiempo | Acción de consola | Evidencia esperada |
|---|---|---|
| 0:00–0:15 | `categoria-crear`: nombre Demo, descripción Defensa; `categorias`. | INSERT y SELECT: aparece el id de categoría. |
| 0:15–0:40 | `producto-crear`: código DEMO-01, nombre Cuaderno, precio 10.00, stock 5, mínimo 1, activo s, categoría anterior, perecedero n. Luego `productos`, buscar DEMO-01. | Producto persistido y búsqueda por código. |
| 0:40–1:00 | `producto-editar`: id creado, mismo código, nombre Cuaderno grande, precio 12.50, stock 5, mínimo 1, activo s, misma categoría, perecedero n. Buscar de nuevo. | UPDATE visible: nombre y precio nuevos. |
| 1:00–1:15 | `producto-borrar` con ese id; buscar de nuevo. | DELETE físico: producto sin ventas deja de aparecer. |
| 1:15–1:30 | `categoria-borrar` con el id de Demo; `categorias`. | Eliminación de categoría ahora sin productos. |

| Tiempo | Acción de consola | Evidencia esperada |
|---|---|---|
| 1:30–1:50 | Intentar `producto-crear` con precio -1; explicar rechazo; abrir CategoriaDao y señalar un parámetro ?. | Validación antes de BD y PreparedStatement. |
| 1:50–2:00 | Explicar capas: vista → controlador → DAO → MySQL. | Comprensión técnica. |

Capturas alternativas: lista inicial, producto creado, producto actualizado, búsqueda tras borrado y mensaje de validación. Añadir captura de SELECT en Workbench si la catedrática requiere ver persistencia independiente.

### 10.1 Demostración adicional de ventas (fuera del video de CRUD)

Crear producto con stock 2; `agregar` cantidad 1; `carrito`; `clientes` para obtener CF; `confirmar` con EFECTIVO; `historial` con fecha de hoy. Verificar stock 1. Intentar cantidad 2 y mostrar rechazo. `producto-borrar` desactiva sin perder el historial. Mostrar prueba `rollbackRevierteSiSegundoProductoFalla` para explicar atomicidad.

### 10.2 Para la defensa

**Explicación:** una demostración debe probar cambios persistentes y reglas, no solo que la aplicación abre. 1. **¿Cómo demuestras UPDATE?** Cambiar un valor y consultarlo nuevamente desde MySQL, no solo modificar una variable. 2. **¿Por qué este producto sí se elimina?** Porque no participa en ventas; uno vendido se desactiva. 3. **¿Qué pasa si escribes letras en stock?** La vista captura la conversión numérica inválida y no llama al DAO con ese dato.

**Cambio en vivo: buscar por nombre.** 1) Ejecutar productos e introducir parte del nombre. 2) Señalar LOCATE con parámetros en ProductoDao.buscarPorTexto. 3) Para búsqueda exacta por código, agregar un método con WHERE p.codigo=? y reutilizar mapear. 4) Probar coincidencia y ausencia sin concatenar entrada en SQL.

## 11 Anexo A — Script de creación de tablas

Ejecutar una sola vez en una base de datos vacía. El script crea seis tablas, restricciones, índices y el registro de Consumidor Final.

```
-- MySQL 8.4 (CHECK también exige MySQL >= 8.0.16). Ejecutar en una BD vacía.
CREATE TABLE usuarios (
 id_usuario INT AUTO_INCREMENT PRIMARY KEY,
 nombre VARCHAR(100) NOT NULL,
 username VARCHAR(50) NOT NULL UNIQUE,
 password_hash VARCHAR(100) NOT NULL,
 activo BOOLEAN NOT NULL DEFAULT TRUE,
 rol VARCHAR(20) NOT NULL,
 CONSTRAINT ck_usuario_rol CHECK (rol IN ('ADMINISTRADOR','CAJERO'))
) ENGINE=InnoDB;
CREATE TABLE categorias (
 id_categoria INT AUTO_INCREMENT PRIMARY KEY,
 nombre VARCHAR(100) NOT NULL UNIQUE,
 descripcion VARCHAR(255) NOT NULL DEFAULT ''
) ENGINE=InnoDB;
CREATE TABLE productos (
 id_producto INT AUTO_INCREMENT PRIMARY KEY,
 codigo VARCHAR(50) NOT NULL UNIQUE,
 nombre VARCHAR(100) NOT NULL,
 precio DECIMAL(12,2) NOT NULL,
 stock INT NOT NULL,
 stock_minimo INT NOT NULL DEFAULT 0,
 activo BOOLEAN NOT NULL DEFAULT TRUE,
 categoria_id INT NOT NULL,
```

```
 tipo VARCHAR(25) NOT NULL,
 fecha_vencimiento DATE NULL,
 CONSTRAINT ck_producto_precio CHECK (precio > 0),
 CONSTRAINT ck_producto_stock CHECK (stock >= 0 AND stock_minimo >= 0),
 CONSTRAINT ck_producto_tipo CHECK (
   (tipo = 'PERECEDERO' AND fecha_vencimiento IS NOT NULL) OR
   (tipo = 'NO_PERECEDERO' AND fecha_vencimiento IS NULL)),
 CONSTRAINT fk_producto_categoria FOREIGN KEY (categoria_id) REFERENCES categorias(id_categoria) ON DELETE RESTRICT ON
     UPDATE RESTRICT
) ENGINE=InnoDB;
CREATE TABLE clientes (
 id_cliente INT AUTO_INCREMENT PRIMARY KEY,
 nombre VARCHAR(100) NOT NULL,
 nit VARCHAR(20) NOT NULL UNIQUE,
 telefono VARCHAR(25) NOT NULL DEFAULT ''
) ENGINE=InnoDB;
CREATE TABLE ventas (
 id_venta INT AUTO_INCREMENT PRIMARY KEY,
 fecha DATETIME NOT NULL,
 usuario_id INT NOT NULL,
 cliente_id INT NOT NULL,
 metodo_pago VARCHAR(10) NOT NULL,
 total DECIMAL(12,2) NOT NULL,
 CONSTRAINT ck_venta_total CHECK (total > 0),
 CONSTRAINT ck_venta_pago CHECK (metodo_pago IN ('EFECTIVO','TARJETA')),
 CONSTRAINT fk_venta_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios(id_usuario) ON DELETE RESTRICT ON UPDATE
     RESTRICT,
 CONSTRAINT fk_venta_cliente FOREIGN KEY (cliente_id) REFERENCES clientes(id_cliente) ON DELETE RESTRICT ON UPDATE
     RESTRICT,
 INDEX ix_venta_fecha (fecha),
 INDEX ix_venta_usuario_fecha (usuario_id, fecha)
) ENGINE=InnoDB;
CREATE TABLE detalle_ventas (
 id_detalle_venta INT AUTO_INCREMENT PRIMARY KEY,
 venta_id INT NOT NULL,
 producto_id INT NOT NULL,
 cantidad INT NOT NULL,
 precio_unitario DECIMAL(12,2) NOT NULL,
 subtotal DECIMAL(12,2) NOT NULL,
 CONSTRAINT ck_detalle_cantidad CHECK (cantidad > 0),
 CONSTRAINT ck_detalle_precio CHECK (precio_unitario > 0),
 CONSTRAINT ck_detalle_subtotal CHECK (subtotal = cantidad * precio_unitario),
 CONSTRAINT uq_detalle_producto UNIQUE (venta_id, producto_id),
 CONSTRAINT fk_detalle_venta FOREIGN KEY (venta_id) REFERENCES ventas(id_venta) ON DELETE RESTRICT ON UPDATE RESTRICT,
 CONSTRAINT fk_detalle_producto FOREIGN KEY (producto_id) REFERENCES productos(id_producto) ON DELETE RESTRICT ON UPDATE
     RESTRICT
) ENGINE=InnoDB;
INSERT INTO clientes (nombre, nit, telefono) VALUES ('Consumidor Final', 'CF', '');
-- No se incluyen contraseñas compartidas: crear administrador con ConsolaApp --crear-admin.
```